SESSION 2

Let's Solve Problems with Arrays, Stacks, Search & Sort

Ameya Ekbote

LAST TIME · QUICK RECAP

What did we actually cover?

Short version: not tricks, not syntax - how to judge whether an approach will survive.

Why DSA still matters

AI can write code, but you still need to review, debug, reason, and choose trade-offs.

Complexity basics

Big-O, growth ladder, constants, lower-order terms, and best / worst / average cases.

Common traps

Half nested loop is still O(n^2). Sequential loops add. n binary searches means O(n log n).

01

How to think when stuck

OUR LOOP FOR EVERY PROBLEM

How we'll solve together

1. Understand

Say the input, output, constraints, and one example in your own words.

2. Brute force

Write the obvious solution first. It gives us correctness and a baseline.

3. Optimize

Let the constraints point us to prefix sum, two pointers, binary search, sorting or stack.

4. Test edges

Empty input, one element, duplicates, sorted/reverse sorted, overflow.

5. Explain the cost

6. Clean up only after it works

02

Today's game plan

We'll keep theory light: quick refresh, tougher problems, coding traps, and plenty of discussion.

1. Array clues
2. Prefix + hash
3. Sort + sweep
4. Stack + binary search

03

Why practice after theory?

Because real questions ask: which idea fits here, and why?

“I know arrays, stacks and sorting. Why do I still get stuck on problems?”

Take a minute: where do you usually get stuck - recall, approach, edge cases, or code?

“I know the topic.
How do I spot the approach?”

Most DSA problems are not about memorizing a trick. They are about reading clues and narrowing choices.

1

Start with constraints

2

Guess the pattern

3

Stress-test edge cases

04

From concept to pattern

“Find the number of subarrays with sum k. Array may contain negative numbers.”

Given an integer array and a target k, return the count of non-empty contiguous subarrays whose sum is k.

Possible solution paths

Step 1/4 - start with the obvious baseline.

Brute force
Try every start and end. Easy baseline, O(n^2) if sums are cached.

Possible solution paths

Step 2/4 - cache range sums, but notice the pair count.

Brute force
Try every start and end. Easy baseline, O(n^2) if sums are cached.

Prefix array
Range sum becomes O(1), but all pairs are still O(n^2).

Possible solution paths

Step 3/4 - tempting pointer idea; useful, but test assumptions.

Brute force
Try every start and end. Easy baseline, O(n^2) if sums are cached.

Prefix array
Range sum becomes O(1), but all pairs are still O(n^2).

Sliding window
Keep left/right pointers over a moving range. Grow/shrink it while tracking sum.

Possible solution paths

Step 4/4 - prefix gives O(1) sums; frequency avoids O(n^2) pairs.

Brute force
Try every start and end. Easy baseline, O(n^2) if sums are cached.

Prefix array
Build once; each range sum O(1).
Still checks O(n^2) ranges.

Sliding window
Keep left/right pointers over a moving range. Grow/shrink it while tracking sum.

Prefix + frequency map
freq[p-k] counts all valid starts for this end in O(1) -> O(n).

Now test the tempting idea

What goes wrong if we try sliding window here?

nums = [1, -1, 1], k = 1

Count the subarrays with sum 1.
Can a grow-and-shrink window count them all?

Why this window needs more care

A negative value changes how the sum moves.

nums = [1, -1, 1], k = 1
Answer: 3 -> [1], [1, -1, 1], [1]

Expanding can decrease the sum; shrinking can increase it.
The usual sum-based window movement rule is unreliable.
Remember earlier prefix sums instead.

THE IDEA

Subarray sum equals k in three steps

```
seen[0] = 1;
for (i = 0; i < n; i++) {
  pref += a[i];
  ans += seen[pref - k];
  seen[pref]++;
}
```

Turn the subarray question into a question about two prefix sums.

1 **Brute force** `O(n^2)`

Enumerate all starts and ends. Good baseline, but it won't survive n = 10^5.

2 **Observe** O(n)

sum(i..j) = pref[j] - pref[i-1]. So for current pref, look for pref - k.

3 **Count while scanning** `O(n) expected`

Store frequencies of previous prefixes. Initialize seen[0] = 1 for subarrays starting at index 0.

Pattern: arbitrary subarray sum + negatives -> prefix sum with a frequency map.

Constraint cheat sheet

LENGTH OF INPUT (n)		TARGET COMPLEXITY
$1 \le n \le 100$	→	$O(n^3)$ and lower · brute force can teach the idea
$100 < n \le 1{,}000$	→	$O(n^2)$ and lower · pair loops, interval comparisons, simple DP
$10^4 \le n \le 10^5$	→	O(n log n) and lower · sort, merge/sweep, binary search on sorted data
$n > 10^5$	→	O(n) and lower · prefix+hash, monotonic stack, window, two pointers
$n > 10^9$	→	O(log answer) around an O(n) check · binary search on answer

NEXT PROBLEM

What survives the collisions?

Input

[10, 2, -5]

Rules

Smaller breaks; equal: both

Return

Survivors, in original order

The problem

Asteroids lie in array order and move at the same speed. Positive goes right; negative goes left. Absolute value is size. Return what remains after all collisions. Values are nonzero.

LET'S DRY RUN IT

One asteroid can trigger a chain

1 Keep 10

Read +10, then +2.
Same direction, same speed.
No collision.
Stack: [10, 2]

2 Read -5

FIRST COLLISION

+2 meets -5.
Size 2 loses to size 5.
Pop +2; -5 is still alive.
Try the new top.

3 Try 10

KEEP GOING

+10 meets -5.
Size 10 wins.
Do not push -5.
Final answer: [10]

Stack = survivors of the processed prefix. Only a positive top and a negative incoming value can collide.

Resolve collisions, then push

```
long long size = -(long long)cur;
while (top>=0 && st[top]>0 && cur<0) {
  if (st[top]<size) {top--; continue;}
  if (st[top]==size) top--;
  alive = 0; break;
}
if (alive) st[++top] = cur;
```

```
[8, -8]           -> []
[5, -10]          -> [-10]
[-2, -1, 1, 2]    -> unchanged
[1, 2, 3, -10]    -> [-10]
```

Time O(n). Extra space O(n).

Each asteroid is pushed at most once and popped at most once. A chain of collisions still gives linear total work.

Try these before running code

Different signs alone are not enough. [-2, 2] move apart. A surviving incoming asteroid can meet several earlier ones.

Setup per asteroid: cur = a[i]; alive = 1. Start top = -1. Return st[0..top].

QUESTION

Count out-of-order pairs

Given an integer array, count pairs (i, j) where i < j and a[i] > a[j].

Each pair of positions counts once. Equal values do not count.

```
a = [2, 4, 1, 3, 5]

Return the number of
out-of-order pairs.
```

Input

An array of integers; duplicates and negative values are allowed.

Return

One count for the whole array. For an empty or one-element array, return 0.

DISCUSSION

Let's think about the pairs

Start with every pair.

```
for (i = 0; i < n; i++)
  for (j = i + 1; j < n; j++)
    if (a[i] > a[j]) count++;
```

Time: O(n^2).

```
What repeats?

For one smaller value, can we count
several earlier bigger values at once?

What would sorted halves give us?
```

Correctness first

Sample answer: 3.
Pairs: (2,1), (4,1), (4,3).

A useful direction

Sorting the entire array loses positions.
Can we count while combining halves?

Next: use the merge step to count crossing pairs.

Solution: count while sorting

Sample answer: 3

Out-of-order pairs:
(2,1), (4,1), (4,3)

```
if (left[i] <= right[j])
  tmp[k++] = left[i++];
else {
  inv += mid - i + 1;
  tmp[k++] = right[j++];
}
```

Start simple

Brute force: check every pair, O(n^2). Correct, but too slow for large n.

Merge-sort count

When right[j] is smaller than left[i], it is smaller than all remaining left values.

Why this works: sorted halves let one comparison count many pairs.

RULES THAT HELP

Pattern cheat sheet

1

Start with constraints

n, value range, negatives, duplicates and update/query counts choose the pattern.

2

Know when window breaks

Negative numbers usually break simple sliding-window sum logic; use prefix + hash.

3

Sort to create structure

Sort unlocks duplicate control, two pointers, binary search and interval reasoning.

4

Stacks store unresolved work

When an element waits for a future smaller/greater element, keep useful candidates only.

5

Binary search needs monotonic yes/no

Before coding, say: if x works, every larger/smaller x also works.

6

Say the invariant first

For pointers, stacks and binary search, the invariant is what makes the solution work.

QUESTION

Remove K Digits

Given a nonnegative number as a string, remove exactly k digits to make the smallest possible number.

Keep the remaining digits in order. Return a string; trim leading zeros.

```
num = "1432219", k = 3

num = "10200", k = 1

Return the smallest number
for each input.
```

The rules

1 <= length <= 100,000; 0 <= k <= length.
If nothing remains, return "0".
Input has no leading zeros, except "0".

DISCUSSION

Which removal helps the most?

Sample answers:
"1432219", k=3 -> "1219"
"10200", k=1 -> "200"

Earlier digits have more influence
on the value than later digits.

```
Keep chosen digits in a stack.
While k > 0 and top > new digit,
pop the top and spend one removal.

If removals remain after scanning,
remove from the end.
```

Start with one removal

Compare keeping "14..." with "13...".
Can a smaller incoming digit replace
a larger digit we have kept?

Test the rule

"12345", k=2 -> "123"
"1111", k=2 -> "11"
Use >, not >=. Trim leading zeros last.

Push once, pop at most once: O(n). Make exactly k removals, then trim leading zeros.

QUESTION

Longest Consecutive Sequence

Given an unsorted integer array, return the length of the longest sequence and the sequence of values that increase by exactly 1.

These values need not be next to each other or appear in order in the input.

```
a = [100, 4, 200, 1, 3, 2]

Output: 4

Sequence: [1, 2, 3, 4]
```

The rules

0 <= n <= 100,000; values within +/-10^9.
Duplicates do not extend a run.
Return 0 for an empty array.

What the example means

The values 1,2,3,4 are all present, even though they occur in different positions.
They form a sequence of length 4.
100 and 200 each stand alone.

HOMEWORK

ToDo before next session :)

Also code the in-class problems. Please check the next two slides for question details.

Q1

Minimum length with negatives

```
int a[] = {2,-1,2,-1,2,3};
int K = 5;
```

Find the minimum length subarray
with sum >= K.
Return -1 if no such subarray.

Q2

Right-side counts with duplicates

```
int a[] = {5,2,6,1,2};
```

For each i, count values on the right
that are strictly smaller than a[i].
Return the answer array.

Shortest subarray reaching K

Given an integer array a and a positive K, return the length of the shortest non-empty contiguous subarray whose sum is at least K.

A subarray uses neighbouring elements without skipping any. Values may be negative. Return -1 if no such subarray exists.

```
a = [2,-1,2,-1,2,3], K = 5
Output: 2

 a = [1,-1,1], K = 4
Output: -1
```

Input and return value

1 <= n <= 100,000; 1 <= K <= 10^9.
Values are integers within +/-10^9.
Return a length, not the sum or indices.

What the examples mean

In example 1, the final [2,3] has sum 5 and length 2. No single value reaches 5.
In example 2, no contiguous subarray has sum at least 4.

Smaller values to the right

Given an integer array a, return an array ans of the same length.

For each position i, ans[i] counts positions j > i where a[j] is strictly less than a[i].
Count each occurrence separately.
Equal values do not count.

```
a = [5,2,6,1,2]
Output: [3,1,2,0,0]

 a = [2,2]
Output: [0,0]
```

Input and return value

0 <= n <= 100,000; values within +/-10^9.
Return counts in the original input order.
For an empty array, return an empty array.

What the examples mean

For 5: smaller right-side values are 2,1,2.
For the first 2: only 1 is smaller.
For 6: both 1 and 2 are smaller.
The final 1 and 2 each have count 0.
In [2,2], equality counts neither value.